


```

Fun(int):
    mov     eax, 42
    mov     edx, 0
    idiv    edi
    ret
main:
    mov     eax, 7
    ret

```

Okay, that looks more like proof that `constexpr` helped before. You now can see the function `Fun`, but the result is still known in `main`. Why is that?

The reason is that `constexpr` implies inline! Try for yourself, make `Fun` `inline`, and you will see exactly the same assembly output as when the function was `constexpr`.

Because of the implicit `inline`, the compiler understands that `Fun` never escapes the current translation unit. By knowing that there is no reason to keep the definition around. Then, `Fun` itself is reasonably simple to the compiler, and the parameter is known at compile-time. An invitation for the optimizer, which it happily accepts.

You can alter the code even more, and the optimizer will still be able to produce the same result. Have a look at the changes I made to the original code in Listing 3.

```

inline auto Fun(int v) ①
{
    return 42 / v;
}

int main()
{
    int val{6};          ②
    auto f = Fun(val);   ③
    return f;
}

```

Listing 3

`Fun` is now inline as ① shows. The input to `Fun` is now a non-const variable `var` ②, and the result of the call to `Fun` in ③ is stored in a non-const variable. All just run-time code. Except that the compiler can still see that the input to `Fun` is always 6. With this knowledge the compiler gets its friend the optimizer onboard and the result is the same as with the initial code that looked way more constant then this version.

What you see here is still an optimization. Yes, if you are interested in a small binary footprint, you will be happy. But, `constexpr` can give you more! You can get guarantees from `constexpr`. Let's explore that.

Ways to enforce constant evaluation

The current code does not force the compiler to evaluate `Fun` at compile-time in a manner that could cause compile-time evaluation to fail. The evaluation could silently fail for integral data types declared `const`, which isn't allowed with `constexpr`. Essentially, you must force the compiler into a compile context for the evaluation. You have roughly four options for doing so:

- assign the result of `Fun` to a `constexpr` variable;
- use `Fun` as a non-type template argument;
- use `Fun` as the size of an array;
- use `Fun` within another `constexpr` function that is forced into constant evaluation by one of the three options before.

In Listing 4, you find the four cases in code.

```

constexpr auto Other(int v)
{
    return Fun(v);
}

int main()
{
    constexpr auto f{Fun(6)};
    int          data[Fun(6)]{}; // Please prefer
                                // the std::array solution
    std::array<int, Fun(6)> data2{};
    constexpr auto      ff{Other(6)};
}

```

Listing 4

Enforcing constant evaluation

So far, I have done neither of the four variants, time to change this. Let me make the variable `f` `constexpr` (Listing 5).

```
constexpr auto Fun(int v)
{
    return 42 / v; ①
}

int main()
{
    constexpr auto f = Fun(6); ②

    return f; ③
}
```

Listing 5

Once you look at the resulting assembly, you see ... no change compared to the initial example. Remember that I started by stating that distinguishing optimization from the guarantee is difficult?

My example now comes with the guarantee that `Fun` is evaluated at compile-time. However, since there is no difference between the former version in the resulting assembly, what is my point?

Well, time to start talking about the guarantee.

What if, and please don't be shocked, I replace `6` with `0` in my call to `Fun`? Urg, yes, that will result in a division by zero. Who, aside from Chuck Norris, can divide by zero? At least, I can't, and neither can any of the compilers I use.

But the initial example, despite the fact that `Fun` is `constexpr`, compiles just fine. Well, this little warning about the division by zero aside. Ah, yes, and the result is, well, potentially the result to expect if one of us could divide by zero.

The guarantee

Make the variable `f` in ③ `constexpr`, or choose another way to force the compiler into constant evaluation. The result? If you make the change, your compile will fail, and the compiler tells you the obvious: a division by zero does not produce a constant value. This is what `constexpr` functions bring you: an evaluation free of undefined behavior!

Putting `constexpr` on a function only gives you a small part of `constexpr`. Only by using a `constexpr` function in a context requiring constant evaluation will you get the full benefits out of it, no undefined behavior.

I hope this article helps you better understand what `constexpr` can offer and how to distinguish the guarantee from a compiler's optimization.

References

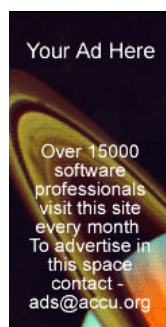
The code listings are available on godbolt:

- Listing 1: <https://godbolt.org/z/chG8oe3TG> (<https://godbolt.org/z/chG8oe3TG>)
- Listing 2: <https://godbolt.org/z/85W5Mdv4T> (<https://godbolt.org/z/85W5Mdv4T>)
- Listing 4 (with the extra needed for it to compile): <https://godbolt.org/z/ddrGrYr3M> (<https://godbolt.org/z/ddrGrYr3M>)
- Listing 5: <https://godbolt.org/z/4Y5nraeYG> (<https://godbolt.org/z/4Y5nraeYG>)

This article was published on **Andreas Fertig's** blog on 6 June 2023, and is available at: <https://andreasfertig.com/blog/2023/06/constexpr-functions-optimization-vs-guarantee/> (<https://andreasfertig.com/blog/2023/06/constexpr-functions-optimization-vs-guarantee/>)

Andreas Fertig is a trainer and lecturer on C++11 to C++20, who presents at international conferences. Involved in the C++ standardization committee, he has published articles (for example, in *iX*) and several textbooks, most recently *Programming with C++20*. His tool – C++ Insights (<https://cppinsights.io> (<https://cppinsights.io>)) – enables people to look behind the scenes of C++, and better understand constructs.

ADVERTISEMENT



(<https://ads.accu.org/www/delivery/ck.php?n=ac1df087&cb=879648978587659>)



([menu-overviews/membership/](https://accu.org/menu-overviews/membership/))